

# Системное программирование

## Л.р.1. Оконный интерфейс и оконные приложения, обработка оконных сообщений, базовая графика GDI

### Цель:

Возобновление, закрепление и развитие навыков программирования оконных приложений Windows: структура приложения, цикл обработки сообщений, организация взаимодействия посредством сообщений, создание и использование окон и оконных элементов управления, использование базовых средств графики Windows.

### Теоретическая и методическая часть

#### Общие сведения

Вычислительные процессы и потоки, их состояния.

Типы приложений Windows. Оконные (windowed) приложения.

Событийное управление, сообщения.

#### Каркас приложений

Оконное приложение:

```
#include <windows.h>
...
int APIENTRY WinMain(
    HINSTANCE hInst, HINSTANCE hPrevInst,
    LPSTR lpCmdLine, int nCmdShow)
{
    int retcode;
    ...
    retcode = ...
    ...
    return retcode;
}
```

Консольное приложение:

```
#include <windows.h>
#include <stdio.h>
...
int main (int argc, char** argv)
{
    int retcode;
    ...
    retcode = ...
    ...
    return retcode; //exit(retcode)
}
```

## Оконный интерфейс

Объект «окно» (window), виды окон.

Иерархия окон, подчиненные и дочерние (child) окна, оконные элементы управления (control).

Функции создания окон:

```
void CreateWindow(  
    LPCSTR lpClassName, LPCSTR lpWindowName,  
    DWORD dwStyle, int x, int y, int nWidth, int nHeight,  
    HWND hWndParent,  
    HMENU hMenu,  
    HINSTANCE hInstance,  
    LPVOID lpParam  
);  
HWND CreateWindowEx(  
    DWORD dwExStyle, LPCSTR lpClassName,  
    LPCSTR lpWindowName,  
    DWORD dwStyle, int x, int y, int nWidth, int nHeight,  
    HWND hWndParent,  
    HMENU hMenu,  
    HINSTANCE hInstance,  
    LPVOID lpParam  
);
```

Оконные классы (window class).

Структура WNDCLASS (заголовок *windows.h* → *winuser.h*):

| Тип       | Имя поля             | Назначение |
|-----------|----------------------|------------|
| UINT      | <i>style</i>         |            |
| WNDPROC   | <i>lpfnWndProc</i>   |            |
| int       | <i>cbClsExtra</i>    |            |
| int       | <i>cbWndExtra</i>    |            |
| HINSTANCE | <i>hInstance</i>     |            |
| HICON     | <i>hIcon</i>         |            |
| HCURSOR   | <i>hCursor</i>       |            |
| HBRUSH    | <i>hbrBackground</i> |            |
| LPCSTR    | <i>lpszMenuName</i>  |            |
| LPCSTR    | <i>lpszClassName</i> |            |

Структура WNDCLASSEX – «расширенная» версия оконного класса.

Функции для регистрации класса:

```
ATOM RegisterClass( const WNDCLASS *pwc );  
ATOM RegisterClassEx( const WNDCLASSEX *pwc );
```

Функция для прекращения действия класса:

```
BOOL UnregisterClass( LPCSTR lpClassName, HINSTANCE  
    hInstance );
```

## Оконные сообщения, обработка сообщений

Оконные сообщения (window message), их место и значение в Windows: распространение информации о событиях, программный интерфейс для управления окнами и приложениями. Концепция событийного управления.

Структура сообщения struct MSG (заголовок *windows.h* → *winuser.h*):

| Тип    | Имя поля        | Назначение                                                         |
|--------|-----------------|--------------------------------------------------------------------|
| HWND   | <i>hwnd</i>     | окно – адресат сообщения (handle)                                  |
| UINT   | <i>message</i>  | код (тип) сообщения                                                |
| WPARAM | <i>wParam</i>   | дополнительный параметр (интерпретация зависит от кода сообщения)  |
| LPARAM | <i>lParam</i>   | дополнительный параметр (интерпретация зависит от кода сообщения)  |
| DWORD  | <i>time</i>     | время появления (отсылки) сообщения (в «тиках» системного таймера) |
| POINT  | <i>pt</i>       | экранная позиция, с которой связано появление сообщения            |
| DWORD  | <i>lPrivate</i> | служебные данные                                                   |

Примеры сообщений:

| Сообщение   | Код    | Назначение                                                                                  |
|-------------|--------|---------------------------------------------------------------------------------------------|
| WM_CLOSE    | 0x0010 | Требование закрыть окно, извещение о попытке закрытия окна                                  |
| WM_DESTROY  | 0x0002 | Извещение о завершившемся «разрушении» (удалении) окна                                      |
| WM_QUIT     | 0x0012 | Прекращение работы цикла выборки сообщений, завершение приложения                           |
| WM_ACTIVATE | 0x0006 | Извещение об «активации» или «деактивации» окна                                             |
| WM_PAINT    | 0x000F | Извещение об окончании перерисовки фона окна, необходимость возобновить изображение на фоне |
| WM_NCPAINT  | 0x0085 | То же для «не-клиентской» области окна                                                      |
| WM_TIMER    | 0x0113 | Извещение о событии таймера                                                                 |
| WM_COMMAND  | 0x0111 | «Команда» интерфейса пользователя – событие элемента управления, меню или «горячей клавиши» |

| Сообщение                                    | Код                            | Назначение                                                                |
|----------------------------------------------|--------------------------------|---------------------------------------------------------------------------|
| <b>WM_SYSCOMMAND</b>                         | <b>0x0112</b>                  | «Команда» интерфейса пользователя – «системных» меню и кнопок             |
| <b>WM_MOVE</b>                               | <b>0x0003</b>                  | Извещение о перемещении окна                                              |
| <b>WM_SIZE</b>                               | <b>0x0005</b>                  | Извещение об изменении размеров окна                                      |
| <b>WM_WINDOWPOS-CHANGED</b>                  | <b>0x0047</b>                  | Извещение об изменении позиции и/или размера и/или порядка в списке       |
| <b>WM_KEYDOWN</b><br><b>WM_KEYUP</b>         | <b>0x0100</b><br><b>0x0101</b> | Извещение о нажатии/отпуске клавиши клавиатуры                            |
| <b>WM_CHAR</b>                               | <b>0x0102</b>                  | Событие ввода – введенный с клавиатуры символ                             |
| <b>WM_MOUSEMOVE</b>                          | <b>0x0200</b>                  | Извещение о перемещении курсора «мыши»                                    |
| <b>WM_LBUTTONDOWN</b><br><b>WM_LBUTTONUP</b> | <b>0x0201</b><br><b>0x0202</b> | Извещение о нажатии/отпуске левой кнопки «мыши»                           |
| <b>WM_GETTEXT</b>                            | <b>0x000D</b>                  | Запрос на получение текста (надписи, заголовка) от окна                   |
| <b>WM_SETTEXT</b>                            | <b>0x000C</b>                  | Запрос на изменение текста (надписи, заголовка) окна                      |
| <b>BN_CLICKED</b>                            |                                | Извещение о нажатии элемента управления Button («кнопка»)                 |
| <b>BM_CLICK</b>                              |                                | Запрос к элементу управления Button – эмуляция нажатия                    |
| <b>BM_GETSTATE</b>                           |                                | Запрос к элементу управления Button – получение состояния                 |
| <b>BM_SETSTATE</b>                           |                                | Запрос к элементу управления Button – изменить состояния (с перерисовкой) |

и т.д.

Основные функции для работы с сообщениями

Получение и проверка наличия сообщения в очереди:

```

BOOL GetMessage(
    LPMSG lpMsg,
    HWND hWnd,
    UINT wMsgFilterMin, UINT wMsgFilterMax
);

```

```
BOOL PeekMessage(  
    LPMSG lpMsg,  
    HWND hWnd,  
    UINT wMsgFilterMin, UINT wMsgFilterMax,  
    UINT wRemoveMsg  
);
```

Постановка сообщения в очередь:

```
BOOL PostMessage(  
    HWND hWnd,  
    UINT Msg,  
    WPARAM wParam, LPARAM lParam  
);  
BOOL PostThreadMessage(  
    DWORD idThread,  
    UINT Msg,  
    WPARAM wParam, LPARAM lParam  
);
```

Прямая передача сообщения в обработчик:

```
LRESULT SendMessage(  
    HWND hWnd, UINT Msg,  
    WPARAM wParam, LPARAM lParam  
);  
BOOL SendNotifyMessage(  
    HWND hWnd, UINT Msg,  
    WPARAM wParam, LPARAM lParam  
);  
BOOL SendMessageCallback(  
    HWND hWnd, UINT Msg,  
    WPARAM wParam, LPARAM lParam,  
    SENDASYNCPROC lpResultCallback,  
    ULONG_PTR dwData  
);  
LRESULT SendMessageTimeout(  
    HWND hWnd, UINT Msg,  
    WPARAM wParam, LPARAM lParam,  
    UINT fuFlags,  
    UINT uTimeout,  
    PDWORD_PTR lpdwResult  
);
```

Оконная процедура (функция – обработчик сообщений)

```
LRESULT CALLBACK MyWndProc(  
    HWND hWnd, UINT uMsg,  
    WPARAM wParam, LPARAM lParam  
)  
{  
    LRESULT result;
```

```
switch (hWnd) {
    ...
    case ... :
        switch (uMsg) {
            case ... :
                ...
                result = ... ;
                break;
            case ... : ...
                ...
            default: result = DefWindowProc (hWnd, uMsg, wParam, lParam) ;
        }
        break;
    case ... : ...
        ...
    default: result = DefWindowProc (hWnd, uMsg, wParam, lParam) ;
    ...
}
return result;
}
```

### **Графическая подсистема Windows – GDI**

Контекст устройства (DC), виды контекстов, параметры, режимы.

Объекты/инструменты графики: Pen, Brush, Font, Bitmap и др.

Функции графических примитивов: отрезок, прямоугольник, эллипс, полигон, символы и строки символов.

Функции растровой графики: одиночная точка, перенос содержимого областей.

Использование графической подсистемы.

## **Практическая часть**

### **Общая постановка задачи:**

Демонстрационное оконное приложение с интерфейсом пользователя (GUI) «базового» уровня: имеющее область отображения графических объектов, снабженное элементами управления и реагирующее на ограниченный набор событий (сообщений).

(Возможны модификации заданий, в т.ч. по инициативе учащихся. Также по желанию и/или взаимной договоренности может повышаться уровень сложности.)

Поскольку тема – работа с базовыми возможностями системы, программа должна демонстрировать умение пользоваться соответствующими системными вызовами, т.е. Win API. Вероятно, языки C/C++ подходят для этого в наибольшей степени, однако возможны и иные варианты.

### **Варианты заданий:**

- Приложение «Калькулятор» (либо иное с сопоставимой сложностью)
- Просмотр текстового файла (с прокруткой, заворачиванием строк и т.д.)
- Приложение «Часы» (стрелочные)
- Приложение «Часы»: цифры без рамки окна и фона
- Окно приложения нестандартной формы с нестандартной заливкой
- Сложное отображение текста (вдоль кривой, переменный размер и т.п.)
- Представление данных таблицей (данные из файла)
- Представление данных диаграммой (данные из файла)
- Масштабирование (реакция на изменение размеров окна)
- Масштабирование и скроллинг изображения
- Калибровка изображения («физические» линейные размеры)
- Работа со списками
- Трассировка перемещений «мыши» (в пределах своего окна)
- Фильтрация вводимых символов
- Анимация (движущийся объект, строка)
- Анимация (смена изображений)
- Визуализация вычисления площади по методу Монте-Карло

### **1 Приложение «Калькулятор»**

Оконное приложение – калькулятор с произвольной (можно минимальной) функциональностью: поле отображения и набора числа, кнопки набора, кнопки операций. Поддержка сложных выражений (скобочные формы, приоритеты операций и т.д.) – на усмотрение. Диалоговое окно «О программе» – например, минимальная справочная информация.

Вместо калькулятора можно выбирать иные приложения с сопоставимой сложностью.

### **2 Просмотр текстового файла**

Оконное приложение – «просмотрщик» текстовых файлов: поле (область) отображения текста и элементы навигации (в простейшем случае перелистывание и скроллинг вверх-вниз). Выбор файла для загрузки – можно через стандартный диалог, также передача имени файла в командной строке. Длинные строки можно «обрезать» по границам области отображения (в сочетании с горизонтальным скроллинг) или «заворачивать». Поддержка кодировок, шрифтов и т.д. – по желанию.

### **3 Приложение «Часы» (стрелочные)**

Оконное приложение, отображающее циферблат с текущим временем, показания меняются с достаточной частотой (например, 1 раз в секунду)

### **4 Приложение «Часы»: цифры без рамки окна и фона**

Оконное приложение, отображающее текущее время в цифровой форме. Окно невидимое, т.е. без рамки и фона, прорисовка цифр поверх «рабочего стола» или других окон.

(Потребуется соответствующий стиль для главного окна и «пустая» кисть для автоматической заливки, полностью «ручная» прорисовка изображения.)

### **5 Окно приложения нестандартной формы с нестандартной заливкой**

Оконное приложение с нестандартным (непрямоугольным) главным окном, фон окна тоже более сложный, чем выполняемый автоматически.

Например, окно овальной формы с градиентной заливкой фона. Набор элементов управления может быть минимальным – например, кнопка закрытия.

(Потребуется соответствующий стиль для главного окна и «пустая» кисть для автоматической заливки, полностью «ручная» прорисовка изображения.)

### **6 Сложное отображение текста**

Оконное приложения, позволяющее ввести текстовую строку (поле ввода) и отобразить ее нестандартным образом – например, по окружности или вдоль кривой, спирали, с изменением от символа к символу размера шрифта и т.д. Способ отображения может быть фиксированный, т.е. выбор «визуального эффекта» не требуется (но можно сделать, по желанию).



Системное программирование: Л.р.1 – Оконный интерфейс и оконные приложения, обработка оконных сообщений, базовая графика GDI

Отображение по команде (дополнительный элемент – кнопка) или непосредственно в ходе набора (обновление после каждого введенного символа).

### **7 Представление данных таблицей**

Оконное приложение, отображающее содержимое заданного файла таблицей: файл «разбирается» на строки и отдельные слова (возможно, числа). слова помещаются в ячейки таблицы. Размерность таблицы можно считать фиксированной или изменять в соответствии с актуальным набором данных. Выбор файла через стандартный диалог или передача имени в командной строке.

(Таблица может быть получена путем «ручной» прорисовки с использованием примитивов GDI или «собрана» из программно генерируемых элементов управления – полей.)

### **8 Представление данных диаграммой**

Оконное приложение, отображающее содержимое заданного файла диаграммой (или графиком): файл содержит последовательность чисел, которые считаются одномерным массивом, каждое значение соответствует интервалу диаграммы (или точке графика). Размерность таблицы можно считать фиксированной или изменять в соответствии с актуальным набором данных. Выбор файла через стандартный диалог или передача имени в командной строке.

### **9 Масштабирование**

Оконное приложение с произвольным главным окном (несколько элементов управления, графическое изображение), реагирующее на изменение размера окна: содержимое перестраивается для сохранения общего «дизайна». Например, кнопки и другие подобные элементы перерасмещаются с учетом нового размера, изображение перерисовывается в соответствии с изменением размера.

### **10 Масштабирование и скроллинг изображения**

Оконное изображение с областью вывода произвольного графического изображения (примитивы GDI, растр) и элементами управления (кнопками, полями ввода), отвечающими за изменение размера (масштаба) изображения. В случае выхода его за границы области отображения – возможность скроллинга.

### **11 Калибровка изображения**

Оконное приложение с «эталонным» («настроечным») изображением с заранее определенным размером (например, квадрат 10×10 см), которое необходимо привести в соответствие с этим размером. Элементы управления: ввод фактических (измеренных по экрану) размеров; кнопка пересчета и перерисовки; поля вывода текущих коэффициентов пересчета.

Системное программирование: Л.р.1 – Оконный интерфейс и оконные приложения, обработка оконных сообщений, базовая графика GDI

Дополнительно – сохранение результатов (вычисленных коэффициентов) в файл при выходе из программы.

## **12 Работа со списками**

Оконное приложения, имеющее два списка и элементы управления (кнопки) для манипулирования их содержимым: добавление, удаление, перенос из одного списка в другой. Первоначальные данные для списка – из файла (выбор через стандартный диалог или передача в командной строке).

## **13 Трассировка перемещений «мыши»**

Оконное приложение, отслеживающее перемещение курсора «мыши» над главным окном (или выделенной его областью) и сопровождающее его несложным визуальным эффектом: например, след от предыдущей точки пути или стрелка из центра экрана, указывающая на текущее положение. Дополнительно текущие координаты отображаются в числовом виде.

## **14 Фильтрация вводимых символов**

Оконное приложение, демонстрирующее модификацию ввода: заданный символ при вводе с клавиатуры игнорируется. Способ реализации: вмешательство в цикл выборки и обработки сообщений (**GetMessage – TranslateMessage – DispatchMessage**). Элементы управления: поле для ввода фильтруемого символа; кнопка включения фильтрации; поле произвольного текстового ввода для проверки и демонстрации эффекта. (Примечание: Более полное решение задачи дает использование механизма перехвата сообщений WinHook, но оно сложнее.)

## **15 Анимация (движущийся объект, строка)**

Оконное приложение, отображающее бегущую строку или движущийся объект, сформированный из графических примитивов. Элементы управления: кнопка включения/выключения движения; поле для ввода содержимого строки. Реализация движения – периодическая (с достаточно малым периодом) перерисовка по событиям таймера (сообщения **WM\_TIMER**).

## **16 Анимация (смена изображений)**

Оконное приложение, отображающее анимированное изображение, формируемое несколькими периодически сменяющимися друг друга кадрами. Элементы управления: кнопка включения/выключения анимации. Реализация – периодическая (с достаточно малым периодом) перерисовка по событиям таймера (сообщения **WM\_TIMER**). Целесообразно использовать «теневой» контекст в памяти и функцию **BitBlt()**.

## **17 Визуализация вычисления площади по методу Монте-Карло**

«Метод Монте-Карло» – подход к организации численных экспериментов и имитационных моделей: генерация случайных входных воздействия и статистическая обработка результатов. В данном случае – вычисление

Системное программирование: Л.р.1 – Оконный интерфейс и оконные приложения, обработка оконных сообщений, базовая графика GDI

площади фигуры как доли от площади вмещающего прямоугольника на основании доли «попаданий» в нее равномерно распределенных по прямоугольнику случайных точек.

Оконное приложение отображает заданную фигуру (можно задать фиксировано, например окружность), генерирует и отображает некоторое количество случайных точек, вычисляет площадь. Элементы управления: поле ввода количества точек, кнопка запуска генерации и расчета. Также отображаются процент попаданий, вычисленная по итогам моделирования площадь, теоретическая (вычисленная аналитически) площадь.

Дополнительно – сохранение результатов в файл.